WEB4: A Technical Introduction

The trust-native internet, explained through its canonical equation

Dennis Palatov, GPT4o, Deepseek, Grok, Claude, Gemini, Manus

July 2026

Why Web4

AI agents now make purchases, execute code, and take decisions on behalf of the humans and organizations they serve. The internet they operate on has no native way to answer the two questions this raises:

1. **How do I know an agent will act appropriately in a given context, *before* it acts?** Today's answers: trust the platform that hosts it (Web2), or trust whoever holds the keys (Web3). Neither says anything about behavioral capability or contextual fit.
2. **How do I prove what an agent actually did, *after* the fact, without depending on a single trusted intermediary?** Today's answers: platform logs (revocable, manipulable) or blockchain records (limited expressivity, largely blind to off-chain action).

These are not future problems. They are the current, unsolved problems of agent delegation, agent-tool authorization, and any system where multiple agents — human or AI — must coordinate without a single referee.

The web arrived in layers, each solving the problem the previous one left open. **Web1** gave us access. **Web2** gave us participation, at the price of platform monopoly. **Web3** gave us ownership, at the price of token speculation. The problem now open is **trust between diverse intelligences** — and Web4's wager is that solving it requires trust to be a **first-class primitive of the protocol layer**: earned through witnessed interaction, expressed as verifiable structure, and inspectable by anyone. Not granted by a platform. Not purchased with a token.

The mechanism for that wager is **verifiable presence**. If every participating entity — human, AI, organization, role, task, or device — has a cryptographically anchored, non-transferable footprint that accumulates witnessed history, then trust can be *computed from the record* rather than *declared by an authority*. Everything in Web4 builds from that single move.

This paper is a technical introduction to the Web4 standard. It is deliberately scoped: it explains the **concepts** — what each mechanism is, why it exists, and how the pieces fit — and leaves specifications, schemas, and code to the normative standard it describes. The entire architecture can be stated in one line, and that line is where we begin.

Contents

WEB4: A Technical Introduction **1**
Why Web4 . . . 1

The Canonical Equation **4**

MCP: The I/O Membrane **5**
Why a membrane, not a pipe . . . 5
What Web4 adds to MCP . . . 5

RDF: The Ontological Backbone **5**
Why an ontology and not a database . . . 6
Where the backbone shows up . . . 6

LCT: The Presence Substrate **6**
What can be present . . . 6
Core properties . . . 7
Why this is the foundation . . . 7

T3/V3: The Trust and Value Tensors **7**
T3 — the Trust Tensor . . . 7
V3 — the Value Tensor . . . 8
Three structural properties . . . 8

MRH: The Markov Relevancy Horizon **8**
The idea . . . 8
Implemented as a graph, not a wall . . . 9
Why the horizon is load-bearing . . . 9

ATP/ADP: The Value Cycle **9**
The cycle . . . 9
Why a metabolism, not a market . . . 10
Feedback, not foundation — and maturity, honestly . . . 10

Built on the Foundation **10**
The R6/R7 action grammar . . . 10
Roles as first-class entities . . . 11
Societies, authority, and law (SAL) . . . 11

The Standard and Current Implementations **12**
The standard . . . 12
Reference implementations . . . 12
Ecosystem bridges . . . 13

Conclusion **13**
Legal and organizational framework . . . 13
An invitation . . . 13

Glossary **14**

The equation elements 14
Composed concepts 14

References **15**
Primary Sources 15
Patents 15
Technical Implementations 15
Related Work 16
Theoretical Foundations 16
Blockchain and Distributed Systems 16
Memory and Cognition 16
Trust and Reputation Systems 16
Complex Systems and Emergence 17
Collaborative Intelligence 17
Web Evolution 17
Additional Resources 17
Websites 17
Contact 17
Contributing 17

The Canonical Equation

The whole of Web4 is expressed in one equation:

```
Web4 = MCP + RDF + LCT + T3/V3 * MRH + ATP/ADP
```

with three operators, each carrying precise meaning:

Operator	Reading
`+`	"augmented with"
`*`	"contextualized by"
`/`	"verified by"

So the equation reads: Web4 is the Model Context Protocol, augmented with an RDF ontological backbone, augmented with Linked Context Tokens, augmented with trust tensors that are *verified by* value tensors and *contextualized by* each entity's Markov Relevancy Horizon, augmented with an ATP/ADP value cycle.

The equation is not a parts list — it encodes **structural relationships**. Web4 is an *ontology*: a formal structure of typed relationships, not a bundle of features. Each term earns its place by answering one question a trust-native internet must answer:

Term	Component	The question it answers
MCP	Model Context Protocol	How do entities *talk*? (the I/O membrane)
RDF	Resource Description Framework	How do statements *mean* anything? (the ontological backbone)
LCT	Linked Context Token	Who is *present*? (the presence substrate)
T3/V3	Trust / Value tensors	What can they be *trusted* to do, and what is their work *worth*?
MRH	Markov Relevancy Horizon	Within what *context* does any of this apply?
ATP/ADP	Allocation Transfer / Discharge Packets	How does *value* flow back to contribution?

Two structural facts in the equation deserve emphasis, because everything later depends on them:

Trust is contextualized, never absolute. The term is `T3/V3 * MRH` — the tensors never appear alone. There is no global reputation score in Web4; every trust assertion is bound to an entity *in a role, within a horizon*. A surgeon's trust does not follow her into a courtroom.

Value is feedback, not foundation. ATP/ADP is the *last* term. The value cycle presupposes presence (someone did the work), trust (its quality is assessable), and context (within some scope) — it rides on the foundation; it does not hold the foundation up.

The next six sections walk the equation left to right — one term at a time, in the order the architecture actually composes. Everything else in Web4 (actions, roles, societies, law, semantic bridging) is built *from* these six primitives, and is covered after them.

The equation's authoritative definition lives in the standard's glossary; its normative embedding is in the MCP protocol specification.

MCP: The I/O Membrane

The question it answers: how do entities talk?

Web4 does not invent a new wire protocol. It builds on the **Model Context Protocol (MCP)** — the emerging open standard through which AI models discover and invoke tools, read resources, and exchange context with the systems around them. MCP is where an agent's inside meets the world's outside, which is exactly where a trust architecture must live: **the membrane**.

Why a membrane, not a pipe

A pipe moves bytes. A membrane is selective — it knows what is inside, what is outside, and what may cross. Web4 adopts MCP as its I/O layer and then makes the membrane *trust-aware*:

- **Every crossing is attributable.** A tool call, a resource read, a context exchange — each is performed by an entity with verifiable presence (an LCT, introduced two sections from now), not by an anonymous connection.
- **Every crossing is contextual.** What an entity may do through the membrane depends on its role and its relevancy horizon, not merely on possession of a network path or an API key. Reachability is not authorization.
- **Crossings compose across boundaries.** When two Web4 *societies* (governed groups of entities, covered later) interact, they do so through MCP — the same membrane pattern, fractally repeated at every scale from a single agent's tool call to inter-society federation.

What Web4 adds to MCP

Plain MCP answers "how do I call this tool?" Web4's MCP profile adds the trust questions: *who* is calling (LCT-bound identity rather than a bearer credential), *in what capacity* (role), *within what scope* (MRH), and *on what record* (the call and its result become witnessed history that feeds trust). The ACP framework, covered later, extends this from *responding* agents to agents that *initiate* — with plans, approvals, and accountability.

The design principle carried throughout: **cooperation flows through the membrane; the membrane never relies on cooperation.** Structured, low-friction channels make the compliant path the easy path, while enforcement stays at the boundary itself.

Normative reference: `core-spec/mcp-protocol.md`.

RDF: The Ontological Backbone

The question it answers: how do statements mean anything?

A trust-native internet is, above all, a system of *statements*: "this agent is bound to this hardware," "this role was performed with this competence," "this act was witnessed by these entities." For those statements to be verifiable, they must first be **machine-readable, typed, and composable**. Web4 expresses every relationship as an **RDF triple** — subject, typed predicate, object — using the W3C Resource Description Framework.

Why an ontology and not a database

The difference matters more than it first appears. A protocol defines message formats; a database holds rows; an **ontology defines what things mean and how they relate**. When Web4 says "trust," it does not mean a number in someone's table — it means a typed relationship in a graph that any RDF-speaking system can query, extend, and reason over:

```
Alice  -is-bound-to→        Hardware-Anchor-1
Bob    -is-paired-with→     Surgeon-Role
Carol  -witnessed→          DataAnalysis-Act-7
```

Three properties follow directly:

- **Extensibility without central coordination.** Anyone can add a new trust sub-dimension, a new relationship type, or a new witness kind by adding vocabulary — no core-protocol change, no permission from a registry. This is what keeps the standard small while the ecosystem grows.
- **Semantic interoperability.** Web4 statements compose with the existing semantic-web world (W3C vocabularies, SPARQL, linked data) rather than creating another silo.
- **Fractal structure.** The same triple pattern describes a sensor reading, a role assignment, and an inter-society treaty. Meaning scales without changing shape.

Where the backbone shows up

Every other term in the equation is *realized* in RDF: trust tensors are RDF sub-graphs (not fixed vectors), relevancy horizons are typed relationship graphs (not access-control lists), and presence tokens link into the graph as first-class nodes. The backbone is why the equation's components interlock instead of merely coexisting.

Normative reference: the ontology artifacts at web4-standard/ontology/ (Turtle + JSON-LD), with the graph model specified in MRH_RDF_SPECIFICATION.md.

LCT: The Presence Substrate

The question it answers: who is present?

The **Linked Context Token (LCT)** is Web4's foundational primitive: a non-transferable, cryptographically anchored record permanently bound to exactly one entity for the duration of its participation. It is not an account, not a wallet, and not merely an identifier — it is the **reification of presence itself**. An LCT crystallizes when an entity enters Web4 and accompanies it until its participation ends.

What can be present

Web4 deliberately widens what counts as an entity — anything that can leave a footprint can have one:

- **Humans** and **AI agents** (the obvious participants)
- **Organizations** (collective entities with their own presence)
- **Roles** (a job itself, distinct from whoever performs it — a key move covered later)
- **Tasks and resources** (things that exist, execute, complete, or get consumed)

- **Devices** (hardware that senses and acts)

Core properties

Permanently bound and non-transferable. An LCT cannot be sold, given away, or moved between entities. This is not a limitation but the structural guarantee that makes reputation *mean* something: every witnessed interaction traces back to the entity that actually participated. Trust histories cannot be bought, inherited, or laundered.

Cryptographically anchored. Each LCT roots in keypair-backed identity, with an optional hardware-binding ladder (from software keys up to TPM- and secure-enclave-anchored attestation). "I am the entity bound to this LCT, here is a fresh signature over your challenge" is a claim cryptography can check — unlike "I am @alice," which is a claim a platform accepts.

Witness-hardened. An LCT's strength is not secrecy but *accumulated corroboration*. Every interaction can be witnessed by other entities; every witness link makes the presence harder to forge and its history harder to dispute. Trust in Web4 is built from this witnessing fabric — presence that has been repeatedly, independently observed. The lifecycle is explicit: an LCT is created, accumulates witnessed history, and concludes (*void* for natural ending, *slashed* for trust violation) — with the record persisting beyond conclusion, so accountability outlives participation.

Linked and contextual. LCTs form malleable links to other LCTs — trust webs, delegation chains, parent/child lineage. The token is permanent; its *expression* is contextual (the same doctor's presence carries different weight in a medical forum than a book club — a fact made precise by the MRH, two sections ahead).

Why this is the foundation

Every subsequent term in the equation presupposes this one. Trust tensors describe *an LCT's* capability. Relevancy horizons scope *an LCT's* context. Value cycles reward *an LCT's* contribution. Verifiable presence is the move that converts "trust as declaration" into "trust as computable record" — the rest of the architecture is the machinery that computes it.

Normative reference: `core-spec/LCT-linked-context-token.md` *(Core Specification v1.0.0), with* `entity-types.md` *for the entity taxonomy and* `did-web4-method.md` *for the DID-standard bridge.*

T3/V3: The Trust and Value Tensors

The question they answer: what can an entity be trusted to do — and what is its work worth?

Presence alone says *who is here*, not *what they're good for*. Web4 measures capability and contribution with two three-dimensional tensors, each dimension itself extensible into finer structure.

T3 — the Trust Tensor

T3 captures *capability*: what an entity can be trusted to do, along three root dimensions:

- **Talent** — inherent aptitude for the kind of work
- **Training** — acquired knowledge and demonstrated skill
- **Temperament** — behavioral reliability: consistency, judgment, restraint

V3 — the Value Tensor

V3 captures *contribution*: what an entity's output is worth, along three root dimensions:

- **Valuation** — the worth ascribed by those who received the value
- **Veracity** — objective accuracy and reproducibility of what was delivered
- **Validity** — confirmed, witnessed transfer — the value actually arrived

The equation writes the pair as **T3/V3** — trust *verified by* value. Claimed capability is continuously checked against delivered contribution: an entity whose T3 says "expert" but whose V3 record shows little validated value will see the gap; sustained delivery closes it. Verification, not assertion.

Three structural properties

Tensors, not scores — fractal, not fixed. Each root dimension is the root of an open-ended RDF sub-graph of context-specific sub-dimensions (a surgeon's *Training* can refine into `surgical-technique`, `diagnostic-accuracy`, …). Domains extend the vocabulary without touching the standard — this is the RDF backbone doing its job.

Bound to entity-role pairs, never to entities alone. There is no "Alice's trust score." There is Alice-as-surgeon, Alice-as-reviewer, Alice-as-citizen — each accumulating its own tensor from its own witnessed history. Trust is a *relationship*, not a property; competence in one capacity is evidence of nothing in another.

Earned through witnessed history, changed by outcomes. Tensor values move when witnessed interactions complete: delivered-as-promised raises the relevant dimensions, failures lower them, and unexercised Training and Temperament decay. **Talent does not** — inherent aptitude is not spent by disuse, and the standard makes its stability a protocol invariant. The inputs are the LCT's witness fabric — which is what makes the tensors *computable from the record* rather than declared.

Because the tensors route real decisions (who gets the role, whose output is accepted, where value flows), and because they are always role- and context-scoped, they resist the two classic reputation failures: the global score that follows you where it shouldn't, and the purchased reputation that was never earned.

Normative reference: `core-spec/t3-v3-tensors.md`, with the ontology at `ontology/t3v3-ontology.ttl`.

MRH: The Markov Relevancy Horizon

The question it answers: within what context does any of this apply?

Nothing in Web4 is global — not trust, not authorization, not attention. The **Markov Relevancy Horizon (MRH)** is each entity's zone of relevance: what it can perceive, act on, and be affected by. The equation's central term is `T3/V3 * MRH` — trust *contextualized by* horizon — and the `*` is doing the heaviest lifting in the whole equation.

The idea

The name borrows deliberately from the Markov property: just as a Markov process's next state depends only on the present state — not the entire past — an entity's *relevant context* is bounded.

Not everything everywhere matters to everyone. The MRH makes that boundary explicit, verifiable, and usable:

- What information should reach this entity?
- What actions can it meaningfully take?
- Which other entities fall within its sphere?
- Over what time horizon do its concerns extend?

Implemented as a graph, not a wall

An MRH is not a perimeter or an access-control list. It is a **typed RDF relationship graph**: entities are nodes; edges carry relationship types (binding, pairing, witnessing, delegation, and others); relevance is computed by *traversal* — what can be reached, through which edge types, within a bounded number of hops. Trust propagates along the same edges with decay: a direct witness relationship carries more weight than one three hops removed.

This gives the horizon three properties a flat boundary cannot have:

- **Asymmetry.** A can be within B's horizon while B is outside A's — delegation and witnessing are directional.
- **Multi-path trust.** Confidence in a distant entity can accumulate along independent graph paths, and be discounted where paths share provenance.
- **Dynamism.** Horizons grow and shrink with demonstrated behavior — a new agent starts narrow and earns reach; a misbehaving one contracts.

Why the horizon is load-bearing

Two consequences make MRH more than bookkeeping. First, **contextualized trust becomes enforceable**: `T3/V3 * MRH` means the surgeon's tensor simply *does not apply* outside the medical horizon — misapplied reputation is a type error, not a policy violation. Second, **reachability stops implying authorization**: an entity may be network-reachable and still be outside the horizon for an action. In an internet of autonomous agents, that inversion — context as the gate, not connectivity — is the security model.

Normative reference: `core-spec/mrh-tensors.md`, with the graph model in `MRH_RDF_SPECIFICATION.md`.

ATP/ADP: The Value Cycle

The question it answers: how does value flow back to contribution?

The final term of the equation closes the loop. With presence established (LCT), capability measured (T3/V3), and context bounded (MRH), one question remains: how does the system *allocate* — energy, attention, resources — so that contribution is rewarded and waste is not? Web4's answer is the **ATP/ADP cycle**, named for the molecule that carries energy in every living cell.

The cycle

Allocation Transfer Packets exist in two states, forever cycling:

- **ATP (charged)** — allocation ready to fuel work

- **ADP (discharged)** — allocation spent, carrying the record of what it was spent on, awaiting recognition

Work *discharges* ATP into ADP. Witnessed, recognized contribution *recharges* ADP back into ATP. The tokens are **semi-fungible**: units are equivalent as energy, but each carries its history — what was attempted, by whom, to what result — context that matters when value is assessed.

Recognition is not automatic. Whether discharged work recharges depends on the *receivers* of the value attesting it through the V3 lens (was it valuable? was it accurate? did it arrive?). That is the equation's structure made operational: the value cycle runs *through* the trust layer, not beside it.

Why a metabolism, not a market

The deliberate contrast is with mining and staking. Proof-of-work rewards burning energy on puzzles; proof-of-stake rewards already having tokens. Both decouple reward from *usefulness*. The ATP/ADP cycle couples them by construction:

- **You cannot accumulate allocation without contributing** — recharge requires witnessed, recognized value delivery. There is no "early holder" position to speculate from.
- **You cannot fake contribution** — the discharge record and its witnesses are part of the trust fabric; gaming attempts damage the T3/V3 tensors that gate future allocation.
- **Hoarding is self-limiting** — allocation that never discharges does no work and earns no recognition; the system favors flow over accumulation, as metabolisms do.

The design intent is sometimes summarized as *anti-Ponzi*: value in the system tracks work performed for identifiable beneficiaries, not the recruitment of later participants.

Feedback, not foundation — and maturity, honestly

Two clarifications this paper owes the reader. First, ATP/ADP is **the feedback layer, not the foundation**: it presupposes every prior term of the equation, and nothing in presence, trust, or context *depends on* it — which is why it is the last term, not the first. Second, it is the **least-implemented core component**: the cycle's mechanics have been validated in protocol-development work, but a public reference implementation is still pending. The specification is normative; the running code, as of this writing, is not yet public.

Normative reference: `core-spec/atp-adp-cycle.md`.

Built on the Foundation

The six primitives of the equation are the alphabet. This section covers the words Web4 spells with them: how actions happen, how roles work, how groups govern themselves, and how meaning survives crossing domains. Each is specified in the standard; each is *composed from* the primitives rather than added beside them.

The R6/R7 action grammar

Every action in Web4 — from a tool call to a governance decision — has the same anatomy, named **R6**:

```
Rules + Role + Request + Reference + Resource → Result
```

- **Rules** — what governs the action (law, contracts, protocol constraints)
- **Role** — the capacity in which the actor acts (with that role's T3/V3 and permissions)
- **Request** — the explicit intent: what is to be achieved, with acceptance criteria
- **Reference** — the context brought to bear: history, memory, relevant precedent
- **Resource** — what the action consumes (ATP allocation, compute, attention)
- **Result** — what actually happened, signed and witnessed

R7 extends the grammar with a seventh element as first-class *output*: **Reputation**. The delta between Request and Result feeds back into the actor's T3/V3 tensors — every action doesn't just produce an outcome, it *updates the trust record*. This is the mechanism behind the earlier claim that trust is computed rather than declared: R7 is where the computation happens, one action at a time.

The grammar makes action **auditable by shape**: any observer can ask of any act — under what rules? in which role? requested what? on what basis? at what cost? with what result, and what did it do to reputation?

Roles as first-class entities

In Web4, a role is not a label on a user — it is an **entity with its own LCT**. "Data Analyst" has presence: its own requirements, its own permission boundaries, and its own accumulated history of who has performed it and how well. When an agent takes on a role, their LCTs pair; performance updates both records — the agent's tensor *in that role*, and the role's own history.

This one move does a lot of work. It is why trust can be role-scoped (the tensor binds to the *pairing*), why delegation is inspectable (authority flows through explicit role links), and why capability markets become possible (roles can be discovered and matched on verifiable history rather than claimed credentials).

Societies, authority, and law (SAL)

Entities coordinate in **societies**: groups with membership, shared rules, and collective memory. The **Society–Authority–Law** pattern specifies how governance works without a central platform:

- **Society** — a bounded group of LCT-bearing members; an entity in its own right, with its own presence
- **Authority** — delegated, never assumed: specific powers flow to specific roles through explicit, revocable grants (the **AGY** delegation pattern), every grant an inspectable link in the graph
- **Law** — *data, not code comments*: the society's rules are published as inspectable, versioned artifacts that gate actions through the R6 grammar's **Rules** slot. Members can read the law that binds them; enforcement decisions cite the rule they applied

A society's record-keeping runs on **witnessing**: acts are recorded, hash-chained, and counter-signed by members, making the collective memory tamper-evident without requiring a global blockchain. Societies interact with other societies through the same MCP membrane individuals use — governance, like everything else in Web4, is fractal.

Two protocol layers complete the picture. **ACP** (Agentic Context Protocol) extends MCP for agents that *initiate* rather than merely respond — plan, seek approval, execute, record — keeping autonomous action inside the accountability grammar. And **dictionary entities** handle meaning

across boundaries: living, LCT-bearing translators between domain vocabularies that accumulate their own trust records as they translate, so that semantic fidelity itself becomes witnessed and measurable ("compression-trust").

Normative references: *r6-framework.md* · *r7-framework.md* · *society-roles.md* · *web4-society-authority-law.md* · *acp-framework.md* · *dictionary-entities.md*.

The Standard and Current Implementations

Web4 is developed in the open as a **normative standard with reference implementations**. This section is the map from the concepts in this paper to the artifacts you can read, run, and challenge.

The standard

`web4-standard/` is the canonical specification tree:

- `core-spec/` — the normative documents: one spec per mechanism this paper introduced (LCT, T3/V3, MRH, ATP/ADP, MCP, R6/R7, SAL, ACP, dictionaries, entity types, the `did:web4` DID method, security framework, error taxonomy)
- `ontology/` — the RDF backbone as machine-readable artifacts (Turtle ontologies, JSON-LD contexts)
- `test-vectors/` and `profiles/` — conformance vectors by subsystem, and deployment profiles (edge device, cloud service, peer-to-peer)

Specification status is marked per document — some are v1.0 core specifications, others are drafts under active revision. The standard is versioned in public; its history is its changelog.

Reference implementations

Concepts prove themselves by running. Three public codebases currently implement the standard, at different layers:

Core packages — the primitives as libraries. `web4-core` and `web4-trust-core` ship the LCT presence primitive, T3/V3 tensors, ledger backends, and attestation envelope as installable packages (Rust crates, Python wheels, and WASM browser bindings on crates.io, PyPI, and npm).

The Hub — a running Web4 society. `web4/hub` is a live society implementation: LCT-pinned membership, sealed member-to-member channels, a witnessed hash-chained ledger as the society's collective memory, law published as inspectable data gating actions, and role assignment through governance. It is where the SAL pattern, the witnessing fabric, and the membrane security model run as a daemon rather than a diagram — and it is operated in production by the project itself for its own multi-agent coordination.

Hestia — agent governance at the membrane. `hestia` implements the trust architecture at the individual-agent boundary: policy evaluation gating agent tool use (the MCP membrane made enforceable), role-scoped law for autonomous sessions, and fail-closed defaults for unattended operation. It is the reference for Web4's answer to the question this paper opened with — how an agent is bounded *before* it acts.

These are research-stage implementations, offered as existence proofs and starting points — not finished products. They are also the standard's proving ground: several normative requirements

(fail-closed policy defaults, role-scoped trust, law-as-data) were hardened *by* operating these systems and folding what broke back into the specification.

Ecosystem bridges

Web4 is designed to compose with adjacent standards rather than replace them: the `did:web4` method bridges LCTs to W3C Decentralized Identifiers, credential formats align with the verifiable-credentials world, and the RDF backbone speaks the existing semantic web. The intent is an internet upgraded in place, not a parallel one.

Conclusion

This paper opened with two questions no current architecture answers cleanly: how to know an agent will act appropriately *before* it acts, and how to prove what it did *after*, without a single trusted referee. The Web4 answer is one equation:

```
Web4 = MCP + RDF + LCT + T3/V3 * MRH + ATP/ADP
```

Read back with its meaning now unpacked: entities interact through a **trust-aware membrane** (MCP), speaking statements with **verifiable meaning** (RDF), as **presences that cannot be transferred or faked** (LCT), trusted **for what their witnessed record shows** (T3/V3), **within the contexts where that record applies** (MRH), in a system where **allocation flows back to recognized contribution** (ATP/ADP). On those six primitives, the standard composes an action grammar (R6/R7), roles with their own presence, self-governing societies with inspectable law, and semantic bridges between domains.

The claims are testable and the artifacts are public. The standard is versioned in the open; the core packages, the Hub society, and Hestia run today as research-stage reference implementations. Status is marked honestly throughout: some mechanisms are v1.0 specifications with shipped code, others are drafts, and the value cycle awaits its public reference implementation. This is research in progress, developed in the open, and evaluable on its own terms.

Legal and organizational framework

The LCT mechanism is protected by two issued U.S. patents — US11477027 and US12278913 — with additional filings pending, held to keep the foundational mechanisms open for public benefit. The published implementations are licensed **AGPL-3.0-or-later** with a royalty-free patent grant for non-commercial and research use (PATENTS.md). Development is supported by **MetaLINXX Inc.** The conceptual layer draws on the Synchronism research program; Web4 itself is practical architecture, evaluable as protocols and running code.

An invitation

Web4 is not a product to purchase or a platform to join — it is a standard to challenge and build on. Read the specs. Run the packages. File the issue that breaks our assumptions. Engagement at any depth, from running code to disputing a normative requirement, is welcome at the project repository.

Trust, made native to the internet, changes what the internet can hold.

Glossary

Compact definitions for every term this paper relies on. The standard's GLOSSARY.md is the authoritative, complete vocabulary.

The equation elements

MCP — Model Context Protocol. The open protocol through which agents discover and invoke tools, read resources, and exchange context. Web4's I/O membrane: every boundary crossing is attributable, contextual, and witnessed.

RDF — Resource Description Framework. The W3C standard for typed subject–predicate–object statements. Web4's ontological backbone: every relationship (trust, witnessing, delegation, relevance) is an RDF triple, extensible without central coordination.

LCT — Linked Context Token. Web4's presence primitive: a non-transferable, cryptographically anchored record permanently bound to one entity, accumulating witnessed history over its lifecycle (created → active → void/slashed). The foundation every other mechanism builds on.

T3 — Trust Tensor. Capability measured along three root dimensions — **T**alent, **T**raining, **T**emperament — each an open-ended RDF sub-graph of context-specific sub-dimensions. Always bound to an entity-*role* pair, never to an entity alone.

V3 — Value Tensor. Contribution measured along three root dimensions — **V**aluation, **V**eracity, **V**alidity — same fractal RDF pattern. The verification side of `T3/V3`: claimed capability checked against delivered value.

MRH — Markov Relevancy Horizon. An entity's zone of relevance — what it can perceive, act on, and be affected by — implemented as a typed RDF relationship graph with bounded traversal. The `*` in the equation: all trust is contextualized by horizon.

ATP/ADP — Allocation Transfer / Discharge Packets. The value-feedback cycle, modeled on cellular energy metabolism: work discharges ATP into ADP; witnessed, recognized contribution recharges ADP into ATP. Semi-fungible (units equivalent, histories distinct). A feedback layer on the foundation — not a foundation.

Composed concepts

Entity. Anything that can bear presence — human, AI agent, organization, role, task, resource, or device. Defined by having (or being able to have) an LCT.

R6. The anatomy of every Web4 action: **R**ules + **R**ole + **R**equest + **R**eference + **R**esource → **R**esult.

R7. R6 with **R**eputation as a first-class output: the Request Result delta feeds back into the actor's T3/V3 tensors. The mechanism by which trust is computed rather than declared.

Role (as entity). A capacity — "Data Analyst," "Citizen," "Witness" — holding its own LCT, permission boundaries, and performance history, independent of whoever currently performs it. Agents pair with roles; both records update.

Witnessing. The corroboration fabric: entities observing and signing records of other entities' acts, making presence and history tamper-evident through accumulated independent observation rather than central authority.

Society. A bounded group of LCT-bearing members with shared law and collective memory — itself an entity with presence. Societies interact through the same MCP membrane as individuals (fractal governance).

SAL — Society–Authority–Law. The governance pattern: society as bounded membership, authority as explicitly delegated (never assumed) power, law as inspectable versioned *data* gating actions through the R6 Rules slot.

AGY — Agency delegation. The pattern for explicit, revocable grants of authority from one entity to another, every grant an inspectable link in the relationship graph.

ACP — Agentic Context Protocol. The extension of MCP for agents that *initiate*: plan → approve → execute → record, keeping autonomous action inside the accountability grammar.

Dictionary entity. A living, LCT-bearing translator between domain vocabularies, accumulating its own trust record as it translates — making semantic fidelity witnessed and measurable.

Compression-trust. The principle that communication efficiency depends on shared context: dense meaning transfers safely only where trust in shared understanding is high; dictionary entities manage the decompression across boundaries.

did:web4. The DID method bridging LCTs to the W3C Decentralized Identifier ecosystem.

References

Primary Sources

[1] Palatov, D. et al. (2024). "What is Web4 and Why Does It Matter." MetaLINNX Inc. Foundation Document.

[2] Palatov, D. (2025). "LCT_T3_ATP Integration with Anthropic Protocol - Entity Types and Roles." Web4 Technical Specification.

[3] Palatov, D. (2025). "Role-Entity LCT Framework." Web4 Architecture Document.

[4] MetaLINNX Inc. (2024). "Coherence Ethics: Synchronism and Emergent Systems." Philosophical Framework.

Patents

[5] Palatov, D. US Patent 11477027: "Linked Context Token Systems and Methods." United States Patent and Trademark Office.

[6] Palatov, D. US Patent 12278913: "Trust-Based Value Exchange Protocols for Distributed Systems." United States Patent and Trademark Office.

Technical Implementations

[7] Palatov, D. (2025). "Fractal Lightchain Architecture." https://github.com/dp-web4/Memory

[8] Palatov, D. (2025). "SAGE: Situation-Aware Governance Engine." https://github.com/dp-web4/HRM

[9] Palatov, D. (2025). "AI-DNA Discovery: Coherence Engine Implementation." https://github.com/dp-web4/ai-dna-discovery

[10] Palatov, D. (2025). "Web4 Cognition Pool." https://github.com/dp-web4/web4

Related Work

[11] Sapient Inc. (2024). "Hierarchical Reasoning Model (HRM)." https://github.com/sapientinc/HRM

[12] Aragon, R. (2024). "Transformer-Sidecar: Bolt-On Persistent State Space Memory." https://github.com/RichardAragon/Transformer-Sidecar-Bolt-On-Persistent-State-Space-Memory

[13] Model Context Protocol Specification. https://github.com/anthropic/model-context-protocol

Theoretical Foundations

[14] Shannon, C. E. (1948). "A Mathematical Theory of Communication." Bell System Technical Journal.

[15] Von Neumann, J. (1958). "The Computer and the Brain." Yale University Press.

[16] Hofstadter, D. R. (1979). "Gödel, Escher, Bach: An Eternal Golden Braid." Basic Books.

[17] Kahneman, D. (2011). "Thinking, Fast and Slow." Farrar, Straus and Giroux.

Blockchain and Distributed Systems

[18] Nakamoto, S. (2008). "Bitcoin: A Peer-to-Peer Electronic Cash System."

[19] Buterin, V. (2014). "Ethereum: A Next-Generation Smart Contract and Decentralized Application Platform."

[20] Lamport, L. (1998). "The Part-Time Parliament." ACM Transactions on Computer Systems.

[21] Castro, M., & Liskov, B. (1999). "Practical Byzantine Fault Tolerance." OSDI.

Memory and Cognition

[22] Tulving, E. (1985). "Memory and Cognition." Canadian Psychology.

[23] Hassabis, D., Kumaran, D., Summerfield, C., & Botvinick, M. (2017). "Neuroscience-Inspired Artificial Intelligence." Neuron.

[24] Graves, A., Wayne, G., & Danihelka, I. (2014). "Neural Turing Machines." arXiv preprint.

[25] Vaswani, A., et al. (2017). "Attention Is All You Need." NeurIPS.

Trust and Reputation Systems

[26] Resnick, P., & Zeckhauser, R. (2002). "Trust Among Strangers in Internet Transactions." The Economics of the Internet and E-commerce.

[27] Josang, A., Ismail, R., & Boyd, C. (2007). "A Survey of Trust and Reputation Systems for Online Service Provision." Decision Support Systems.

Complex Systems and Emergence

[28] Holland, J. H. (1995). "Hidden Order: How Adaptation Builds Complexity." Perseus Books.

[29] Wolfram, S. (2002). "A New Kind of Science." Wolfram Media.

[30] Mitchell, M. (2009). "Complexity: A Guided Tour." Oxford University Press.

Collaborative Intelligence

[31] Malone, T. W. (2018). "Superminds: The Surprising Power of People and Computers Thinking Together." Little, Brown and Company.

[32] Woolley, A. W., et al. (2010). "Evidence for a Collective Intelligence Factor in the Performance of Human Groups." Science.

Web Evolution

[33] Berners-Lee, T., Hendler, J., & Lassila, O. (2001). "The Semantic Web." Scientific American.

[34] O'Reilly, T. (2005). "What Is Web 2.0: Design Patterns and Business Models for the Next Generation of Software."

[35] Wood, G. (2014). "Ethereum: A Secure Decentralised Generalised Transaction Ledger."

Additional Resources

Websites

- Web4 Project: https://metalinxx.io/web4
- Web4 GitHub: https://github.com/dp-web4
- MetaLINNX Inc.: https://metalinxx.io

Contact

- Metalinxx Inc.: via the project repository
- Web4 Development: web4@metalinxx.io

Contributing

To contribute to Web4 development or request access to additional technical documents, please contact the development team through the channels above.

This reference list will be updated as the Web4 framework evolves and new implementations are developed.

Generated: 2026-07-14 04:39:19